

CREDTRACK AI

for Credit Card and Bill Management System

Course: CSYE 7374 — Intro AI Agent Infrastructure

Professor: Yusuf Ozbek

College: Northeastern College of Engineering

Semester: Spring 2026

Student: Siddharth Yogesh Nashikkar

1. Introduction and Motivation

The CredTrack AI Agent is a distributed, event-driven backend system that monitors a user's Gmail inbox for credit card statements and utility bills, uses a locally-hosted Large Language Model to extract all structured financial data from unstructured email text, and persists everything through an event-sourced actor pipeline — all without any manual input from the user. The system is built on Akka Typed 2.6.21 (Java), Spring Boot 3.3.5, Spring AI 1.0.0, and Ollama running Llama 3.1.

The motivation is straightforward: a user with four credit cards across Chase, Bank of America, Discover, and American Express receives twelve or more statement emails a month, each formatted differently. Manually tracking balances, due dates, and individual transactions across all of them is error-prone. CredTrack solves this by treating the Gmail inbox as a real-time financial data source and using an LLM as the universal parser that can handle any bank's email format without rule-based templates.

This project directly demonstrates every major topic covered in CSYE 7374: LLM integration via Spring AI, structured output extraction with prompt engineering, the Akka Actor Model for concurrent message-oriented processing, fault tolerance with supervision and backoff strategies, Akka Cluster with shard-based distribution, Akka Persistence for event sourcing, and the integration of LLM calls inside a live actor mailbox using the pipeToSelf pattern.

2. Technology Stack

- Java 21 — virtual threads (`Executors.newVirtualThreadPerTaskExecutor()`) for all blocking I/O
- Spring Boot 3.3.5 — REST API layer; internal endpoints for actor-to-backend writes
- Akka Typed 2.6.21 (Apache 2.0) — typed actor system, cluster sharding, persistence
- Spring AI 1.0.0 — provider-abstracted LLM client (Ollama backend)
- Ollama — local LLM runtime; serves Llama 3.1 at `http://localhost:11434`
- Llama 3.1 — the LLM model used for all extraction (statement, transaction, utility bill)
- Google Gmail API v1 — History API for incremental email polling
- PostgreSQL — primary data store for users, cards, statements, transactions, utility bills
- Jsoup — HTML stripping and text normalization on email bodies before LLM submission
- Akka Persistence (in-memory journal + filesystem snapshots) — event sourcing

3. LLM Integration — Spring AI, Ollama, and Llama 3.1

The entire intelligence of the system sits in `ExtractionService` — a Spring-managed service that wraps the Spring AI `ChatClient` and exposes three typed extraction methods: one for credit card statements, one for individual transaction alert emails, and one for utility bills. The LLM is never called directly; Spring AI's abstraction layer means the underlying model can be swapped from Ollama to any other provider (OpenAI, Anthropic) with a single configuration change.

3.1 Spring AI ChatClient Configuration

application.properties configuration:

```
spring.ai.ollama.base-url=http://localhost:11434
spring.ai.ollama.chat.options.model=llama3.1
spring.ai.ollama.chat.options.num-predict=1024
spring.ai.ollama.chat.options.format=json
```

The `format=json` option instructs Ollama to constrain its output to valid JSON, which eliminates the most common failure mode (LLM wrapping output in markdown code blocks). The `ChatClient` is injected via Spring AI's auto-configuration:

```
public ExtractionService(ChatClient.Builder chatClientBuilder) {
    this.chatClient = chatClientBuilder.build();
}
```

All three extraction methods call the same underlying pattern:

```
String raw = chatClient.prompt()
    .user(prompt)
    .call()
    .content();
```

The raw string is then parsed with a regex that extracts the first complete JSON object from the response: `Pattern.compile("\\[{\\s\\S}*", DOTALL)`. This tolerates any stray text the model places before or after the JSON.

3.2 Email Pre-Processing Before LLM Submission

Before any email body is sent to the LLM, `ExtractionService` normalises it:

- HTML tags are stripped using `Jsoup.parse(body).text()` — leaving plain readable text
- Whitespace is collapsed (multiple spaces and newlines reduced to single space)
- The text is truncated to a maximum of 4,000 characters to fit the model's context window
- Statement and payment URLs (`viewStatementUrl`, `makePaymentUrl`) are extracted separately from HTML before stripping and appended after the truncated text — ensuring they are never lost in truncation

3.3 Statement Extraction — Prompt and Output Schema

The statement extraction prompt is bank-aware: the bank key (e.g. CHASE, AMEX, BOA) is injected into the prompt so the model understands the email's origin. The prompt instructs the LLM to return a JSON object with these fields:

- `isStatement` (boolean) — true if this email is a credit card statement
- `cardDigits` (string) — last four digits of the card as printed in the email
- `statementBalance` (number) — the closing balance of the statement
- `minimumPaymentDue` (number) — the minimum payment amount
- `statementDate` (string) — the statement closing date
- `dueDate` (string) — the payment due date
- `viewStatementUrl` (string) — link to the PDF or online statement
- `makePaymentUrl` (string) — link to the payment portal
- `confidence` (number 0–1) — the model's self-reported extraction confidence

Confidence threshold: 0.7 — statements with confidence below this are discarded and not written to the database.

3.4 Transaction Extraction — Prompt and Output Schema

Transaction alert emails (e.g. "A purchase of \$86.67 was made at Costco") are processed by a separate extraction method with its own prompt. The LLM returns:

- `isTransaction` (boolean) — true if this is a transaction alert
- `merchantName` (string) — the merchant as named in the email
- `merchantCategory` (string) — category inferred by the LLM (GROCERY, DINING, TRAVEL, etc.)
- `amount` (number) — the transaction amount
- `currency` (string) — always USD for supported banks
- `transactionDate` (string) — the date of the purchase
- `transactionType` (string) — PURCHASE, PAYMENT, or CREDIT
- `description` (string) — any additional detail from the email
- `confidence` (number 0–1) — self-reported confidence

Confidence threshold: 0.90 — intentionally higher than statements because transaction alerts are shorter and more ambiguous; false positives are more damaging.

3.5 Utility Bill Extraction

Utility bill emails from Eversource and National Grid are extracted by a third prompt variant. URL extraction is skipped for bills (utility emails tend to have fewer links and appending them causes hallucination). The LLM returns:

- `isBill` (boolean) — true if this is a utility bill email
- `accountLastFour` (string) — last four digits of the utility account number
- `amountDue` (number) — the amount owed
- `dueDate` (string) — the payment due date
- `billDate` (string) — the bill issue date

- billingPeriodStart / billingPeriodEnd (string) — the billing window covered
- confidence (number 0–1)

4. Actor Model Design — Akka Typed

The entire asynchronous pipeline is built with Akka Typed actors. Each actor is a self-contained unit of computation with its own private state, communicating only through immutable typed messages. No shared mutable state exists anywhere in the system. Blocking I/O (Gmail API calls, LLM calls, REST calls to the backend) is always dispatched to a Java 21 virtual thread executor and piped back into the actor's mailbox via `pipeToSelf` — the actor thread is never blocked.

4.1 Actor System Root

The `ActorSystem` is named "credtrack-ai" and is bootstrapped by `AkkaConfig.java` (Spring `@Configuration`). It directly supervises the following top-level actors:

- `EmailPipelineCoordinator` — root of the statement/transaction polling pipeline
- `UtilityBillCoordinatorActor` — root of the utility bill polling pipeline
- `PersistentStatementLedgerActor` — event-sourced ledger (supervised with `restartWithBackoff`)
- `AnalyticsAggregatorActor` — computes unbilled spend via `Ask` pattern to the ledger
- `AnalyticsAgentHolder` — lock-free `AtomicReference`-backed analytics snapshot store
- `TransactionInitActor` — spawned per poll cycle for transaction backfill
- `ClusterSharding(AnalyticsWorkerActor)` — sharded per `userId` across the cluster

4.2 Statement Pipeline — Actor Breakdown

EmailPipelineCoordinator

The root coordinator for all credit card data. Receives a `Poll` command every hour from `EmailPollScheduler`. On `Poll`, it pipes a `CredentialsFetch` to the backend (`pipeToSelf`) and on `CredentialsFetched` checks each user: if `gmailScanComplete=false`, it spawns `InitCardScanActor` for historical backfill; otherwise it spawns `EmailFetcherActor` for incremental polling. An `activeUsers` set provides mutual exclusion — the same user is never processed in two concurrent cycles.

- `Poll` — triggers the cycle
- `CredentialsFetched(users)` — response from backend credential fetch
- `FetchComplete(userId, newHistoryId, llmFailed)` — from `EmailFetcherActor` on completion
- `InitScanComplete(userId) / InitScanFailed(userId)` — from `InitCardScanActor`
- `TransactionBackfillComplete(userId)` — from `TransactionInitActor`

InitCardScanActor

A one-shot actor spawned per new card. It performs a historical keyword search of Gmail for all past statement and payment emails for that card. Statement emails are searched and dispatched to `StatementExtractorActor` in parallel (multiple LLM calls run concurrently). Payment emails are processed sequentially by `PaymentExtractorActor` to prevent race conditions on the same card

record. Once all cards in the batch are processed, the coordinator is notified via `InitScanComplete` and the card's `gmailScanComplete` flag is set to true via `POST /internal/cards/{cardId}/init-complete`.

Search query examples:

```
Chase statements:    from:chase.com subject:statement <lastFour>
BOA statements:      from:bankofamerica.com subject:statement <lastFour>
Amex statements:     from:americanexpress.com subject:statement
Chase payments:      from:chase.com "payment is scheduled" <lastFour>
BOA payments:        from:eaalerts.bankofamerica.com "received your credit
card payment" <lastFour>
```

EmailFetcherActor

Handles incremental polling for cards that have already been initialised. Calls the Gmail History API with the stored `historyId` cursor to retrieve only messages added since the last poll. Each matched email is dispatched to `StatementExtractorActor` or `PaymentExtractorActor` based on subject-line pattern. Tracks a `llmFailed` flag: if any LLM extraction reports failure, the flag is set and `FetchComplete` is sent with `llmFailed=true`, preventing the coordinator from advancing the `historyId` cursor.

StatementExtractorActor

A short-lived actor (one per email) that calls `ExtractionService.extract(bankKey, body)` via `pipeToSelf`. On `ExtractionDone`, it validates the extracted card digits against the stored card's `lastFour` — a mismatch sets `status=WRONG_STATEMENT`. Passes the validated result to `StatementWriterActor` on success; reports `ExtractionFailed` on LLM error or confidence below 0.7.

PaymentExtractorActor

Uses regex patterns (not LLM) for payment confirmation emails, which are highly structured and bank-specific. Supports Chase, Bank of America, Discover, and Amex patterns. On successful parse, POSTs to `/internal/statements/{id}/mark-paid` via a `pipeToSelf` HTTP call. Stops itself after one parse cycle.

StatementWriterActor

Receives validated `StatementExtraction` results and POSTs them to `POST /internal/statements` on the Spring Boot backend. The `gmail_message_id` unique constraint in the database ensures idempotency — retries never create duplicate statements. On `WriteDone`, notifies `PersistentStatementLedgerActor` to record the new closing date.

4.3 Transaction Pipeline

TransactionInitActor

After a card's initial scan completes, `TransactionInitActor` performs a targeted backfill of transaction alert emails covering two billing cycles: the current unbilled period and the previous month.

Supported banks: Chase, BOA, Discover, Citi, Capital One, Wells Fargo, US Bank. Amex is excluded (uses weekly statement snapshots rather than individual transaction alerts).

TransactionRouterActor + LlmTransactionExtractorActor

TransactionRouterActor receives RouteTransaction messages and forwards them to LlmTransactionExtractorActor. The router is supervised with restartWithBackoff (1 s → 30 s, randomFactor=0.2) catching RuntimeException. LlmTransactionExtractorActor calls ExtractionService.extractTransaction() with the 0.90 confidence threshold and posts confirmed transactions to POST /internal/transactions.

4.4 Utility Bill Pipeline

The utility pipeline is a structural mirror of the statement pipeline with key differences:

- UtilityBillCoordinatorActor coordinates bill polling; activeAccounts set for mutual exclusion
- UtilityBillInitActor: historical lookback is 180 days (vs. unlimited for cards)
- Incremental polling uses a fixed 45-day lookback window (no historyId — utility providers send fewer emails)
- UtilityBillRouterActor uses nested supervision: IOException → restart (1s→30s), IllegalArgumentException → stop
- UtilityBillExtractorActor: LLM extraction with no URL injection to prevent hallucination
- UtilityPaymentExtractorActor: regex parsing for Speedpay payment confirmation emails
- Search queries target Eversource (notifications.eversource.com) and National Grid (nationalgridus.com) sender domains
- Backend endpoint: POST /internal/utility-bills instead of /internal/statements

4.5 Complete Actor Hierarchy

The full supervised tree:

```
ActorSystem [credtrack-ai]
├─ PersistentStatementLedgerActor [restartWithBackoff 2s→60s, jitter=0.1]
├─ AnalyticsAgentHolder [restart() on any Exception]
├─ AnalyticsAggregatorActor
├─ EmailPipelineCoordinator
│   └─ StatementWriterActor
│       └─ InitCardScanActor (spawned per poll per unscanned user)
│           └─ StatementExtractorActor (parallel, one per statement email)
│               └─ PaymentExtractorActor (sequential, one per payment email)
│                   └─ EmailFetcherActor (spawned per poll per scanned user)
│                       └─ StatementExtractorActor (per matched statement email)
│                           └─ PaymentExtractorActor (per matched payment email)
└─ TransactionInitActor (spawned per poll)
```

```

|       └─ TransactionRouterActor           [restartWithBackoff 1s→30s,
jitter=0.2]
|       └─ LlmTransactionExtractorActor
└─ UtilityBillCoordinatorActor
|   └─ UtilityBillInitActor                 (spawned per poll per unscanned
account)
|   |   └─ UtilityBillExtractorActor
|   |   └─ UtilityPaymentExtractorActor
|       └─ UtilityBillRouterActor           [IOException→restart 1s→30s |
IllegalArg→stop]
|       └─ UtilityBillExtractorActor
|       └─ UtilityPaymentExtractorActor
└─ ClusterSharding
    └─ AnalyticsWorkerActor[userId] (one shard per user)

```

5. Integrating LLM Calls with Akka — the pipeToSelf Pattern

The fundamental challenge of combining LLMs with Akka actors is that LLM inference is a blocking HTTP call that can take several seconds, but actors must never block their thread — a blocked actor cannot process any other messages, defeating the concurrency model entirely.

The solution used throughout this system is `pipeToSelf`: the actor wraps the blocking call in a `CompletableFuture`, dispatches it to the Java 21 virtual thread executor, and registers a callback that routes the result back into the actor's own typed mailbox as a message. The actor's thread is free to process other messages while the LLM call runs on a virtual thread.

The pattern in `StatementExtractorActor`:

```

// Inside onExtract() — the actor dispatches the LLM call and returns
immediately

CompletableFuture<StatementExtraction>           future           =
CompletableFuture.supplyAsync(
    () -> extractionService.extract(bankKey, emailBody),
    virtualThreadExecutor
);

getContext().pipeToSelf(future, (result, ex) -> {
    if (ex != null) return new ExtractionFailed(messageId,
ex.getMessage());
    return new ExtractionDone(messageId, result);
});

```

This pattern is used in every actor that performs I/O: Gmail API calls in `EmailFetcherActor`, LLM calls in `StatementExtractorActor` and `LlmTransactionExtractorActor`, HTTP backend calls in

StatementWriterActor and PaymentExtractorActor, and analytics data fetches in AnalyticsWorkerActor. Java 21 virtual threads make the executor essentially free — thousands of concurrent LLM calls can be in-flight simultaneously without blocking OS threads.

6. Fault Tolerance and Supervision

Every long-running actor in the system is protected by a supervision strategy. Akka Typed uses a declarative model: the parent actor specifies how to respond to child failures (restart, stop, or escalate) via a SupervisorStrategy. Short-lived per-email actors are not supervised — they are allowed to stop on failure and the coordinator is notified via the ExtractionFailed message, which causes the historyId not to advance.

6.1 Supervision Strategies Applied

- PersistentStatementLedgerActor — restartWithBackoff(minBackoff=2s, maxBackoff=60s, randomFactor=0.1): the event journal is durable so crash recovery is safe; jitter prevents thundering herd on restart
- TransactionRouterActor — restartWithBackoff(minBackoff=1s, maxBackoff=30s, randomFactor=0.2) on RuntimeException: router can fail on bad message routing; short backoff keeps throughput high
- UtilityBillRouterActor — nested strategy: IOException → restart(1s→30s) (transient network error), IllegalArgumentException → stop (bad message format, not retryable)
- AnalyticsAgentHolder — restart() on any Exception: analytics state is recomputable from the backend on restart; no data loss risk

6.2 historyId Suppression as Fault Tolerance

Beyond actor supervision, the system has an application-level fault tolerance mechanism for LLM failures. EmailFetcherActor tracks a llmFailed boolean during each polling cycle. If any StatementExtractorActor reports ExtractionFailed, the flag is set. When the cycle completes, FetchComplete is sent to the coordinator with llmFailed=true. The coordinator then does NOT call backendApiClient.updateHistoryId() — the cursor stays at its previous position. On the next hourly poll, the same emails are fetched again and re-processed. Combined with the gmail_message_id unique constraint in PostgreSQL (which prevents duplicate statement inserts), this guarantees that no statement is permanently lost due to a transient LLM outage.

6.3 Idempotency by Design

The StatementWriterActor POSTs to /internal/statements. The backend's database has a unique constraint on gmail_message_id in the card_statements table. If a statement is written successfully and then the system crashes before advancing the historyId, the next cycle re-sends the same data — the backend returns a conflict (409) and the actor treats it as WriteDone. No duplicates are created regardless of how many times the same email is processed.

7. Akka Cluster and Shard-Based Distribution

7.1 Cluster Configuration

The cluster is configured in application.conf:

```
akka {
  actor.provider = cluster
  cluster {
    seed-nodes = ["akka://credtrack-ai@127.0.0.1:2551"]
    downing-provider-class = "akka.cluster.sbr.SplitBrainResolverProvider"
  }
  remote.artery {
    transport = tcp
    canonical.hostname = "127.0.0.1"
    canonical.port      = 2551
  }
}
```

The seed node at port 2551 is the entry point for cluster membership. The system is fully cluster-aware: actor.provider=cluster, ClusterSharding is initialised, and SplitBrainResolverProvider is configured to handle network partitions by automatically downing the minority side. The architecture is designed to scale horizontally — additional nodes join the cluster by referencing the same seed address, with no code changes required.

7.2 ClusterSharding — AnalyticsWorkerActor

Sharding is initialised in AkkaConfig.java:

```
ClusterSharding sharding = ClusterSharding.get(system);
sharding.init(Entity.of(
    AnalyticsWorkerActor.TYPE_KEY, // EntityTypeKey<Command> "analytics-
    worker"
    ctx -> AnalyticsWorkerActor.create(
        ctx.getEntityId(), // entityId = userId
        backendApiClient,
        virtualThreadExecutor
    )
));
```

The entity ID is the userId string. Akka ClusterSharding maps each userId to exactly one shard region using consistent hashing, distributing actor instances across the cluster automatically. Each user's AnalyticsWorkerActor processes that user's analytics in complete isolation — a failure in one user's shard has no effect on any other. As nodes are added to the cluster, Akka rebalances shards transparently with no code changes required.

7.3 AnalyticsWorkerActor — What It Computes

The sharded actor computes and caches two types of analytics per user (TTL: 15 minutes):

Card Spending Analytics

- Fetches RawCardStatementData from the backend (monthly grouped statement data)
- Computes total spend and total transaction count per card
- Builds a monthly breakdown array showing spend per card per month
- Returns CardSpendingData: totalSpend, totalTransactions, cards[], monthlyBreakdown[]

Utility Bill Analytics

- Fetches RawUtilityData from the backend (all bills per utility account)
- Groups bills by billerName | accountLastFour composite key
- Computes per-account: list of bills (sorted by date), averageAmount, latestAmount
- Computes percentChange vs previous bill if 2+ data points exist
- Returns UtilityData: accounts[] each with full bill history and trend stats

8. Event Sourcing with Akka Persistence

8.1 PersistentStatementLedgerActor

PersistentStatementLedgerActor is an EventSourcedBehavior — an Akka actor that writes every state change to a persistent event journal before applying it to its in-memory state. If the process crashes, Akka replays the full event journal on restart and the actor recovers its exact state with zero data loss.

PersistenceId: "statement-ledger" (single global instance, not sharded)

Journal: akka.persistence.journal.inmem (in-memory journal, reset on process restart)

Snapshots: Stored to local filesystem at target/snapshots/

8.2 Events, State, and Commands

Events persisted to the journal:

```
StatementClosedEvent(userId: String, userCardId: Long, statementDate:
LocalDate)
```

State maintained in memory:

```
LedgerState(latestDates: Map<Long, LocalDate>)
```

Maps each userCardId to its most recent statement closing date. State is updated only if the incoming date is later than the stored one (idempotent replay).

Commands:

- RecordStatementClosed(userId, userCardId, statementDate) — persists StatementClosedEvent, then updates state

- `QueryLatestStatementDate(userCardId, replyTo)` — reads state synchronously, replies with the stored date (or null)

8.3 How the Ledger is Used

- `StatementWriterActor` — sends `RecordStatementClosed` after every successful backend write, keeping the ledger current
- `EmailPipelineCoordinator` — seeds the ledger at the start of each poll cycle with current backend data
- `AnalyticsAggregatorActor` — uses Ask pattern to send `QueryLatestStatementDate` and receives the closing date to compute the unbilled spend window

8.4 Crash Recovery Flow

1. Process crashes at any point during extraction or writing
2. `ActorSystem` restarts; `AkkaConfig` recreates `PersistentStatementLedgerActor`
3. `Akka Persistence` replays all `StatementClosedEvents` from the journal in order
4. `LedgerState` is rebuilt: `latestDates` map is repopulated for every `userCardId`
5. Actor resumes processing — no manual recovery or data correction required
6. Since `historyId` was not advanced (LLM failure prevention), the same emails are re-fetched and re-processed on the next poll

9. Gmail Integration — History API and Incremental Polling

9.1 Why the History API

The Gmail History API (`users.history.list`) returns only messages added since a given `historyId` cursor — it does not scan the entire inbox on every poll. This makes polling both fast (typically 0–5 emails per cycle) and quota-efficient (the Gmail API has a daily quota; full inbox scans would exhaust it quickly). The `historyId` acts as a durable watermark: it advances only after all emails in the current batch have been successfully processed.

9.2 The Poll Cycle — Step by Step

7. `EmailPollScheduler` fires every 3,600,000 ms (1 hour); sends `Poll` to `EmailPipelineCoordinator`
8. Coordinator fetches all user credentials from the backend via `pipeToSelf`
9. For each user: if `gmailScanComplete=false` → spawn `InitCardScanActor`; else → spawn `EmailFetcherActor`
10. `EmailFetcherActor` calls:
`gmail.users().history().list(USER).setStartHistoryId(historyId).setHistoryTypes(["messageAdded"]).execute()`
11. Response is filtered: only `messageAdded` entries, only messages from known bank sender domains
12. First-time users (`historyId=null`): record current position via `gmail.users().getProfile(USER).getHistoryId()`; return empty (`InitCardScanActor` handles history)

13. 404 responses (deleted messages) are skipped gracefully
14. Each matched email is dispatched: statement subject patterns → StatementExtractorActor, payment patterns → PaymentExtractorActor
15. On success: FetchComplete(userId, newHistoryId, llmFailed=false) → coordinator advances cursor via POST /internal/credentials/{userId}/history-id
16. On any LLM failure: FetchComplete(userId, newHistoryId, llmFailed=true) → coordinator does NOT advance cursor → same emails retried next hour

9.3 Historical Backfill — InitCardScanActor

When a user adds a new card, InitCardScanActor runs a targeted keyword search of the full Gmail inbox (not history) to recover all past statements:

```
gmail.users().messages().list(USER)
    .setQ("from:chase.com subject:statement 5135")
    .execute()
```

The search queries are bank-specific and include the card's last four digits (except Amex, where the 4-digit card number causes tokenization issues so digits are omitted). Statement emails are processed in parallel (multiple StatementExtractorActor instances running concurrently). Payment emails are processed sequentially to prevent concurrent updates to the same card record.

10. Key Configuration Reference

- Server port: 8081
- Ollama base URL: http://localhost:11434
- LLM model: llama3.1
- LLM max tokens (num-predict): 1,024 per call
- LLM output format: json (forced by Ollama config)
- Email body max chars before LLM: 4,000 characters
- Statement confidence threshold: 0.7
- Transaction confidence threshold: 0.90
- Gmail poll interval: 3,600,000 ms (1 hour)
- Transaction scan interval: 1,440 minutes (24 hours); overridable via TRANSACTION_SCAN_INTERVAL_MINUTES env var
- Analytics cache TTL: 15 minutes (AnalyticsWorkerActor)
- Utility bill init lookback: 180 days
- Utility bill incremental lookback: 45 days
- PersistentStatementLedgerActor restartWithBackoff: min=2s, max=60s, randomFactor=0.1
- TransactionRouterActor restartWithBackoff: min=1s, max=30s, randomFactor=0.2
- UtilityBillRouterActor: IOException→restart(1s→30s), IllegalArgumentException→stop
- Executor: Java 21 virtual threads (Executors.newVirtualThreadPerTaskExecutor())
- Akka cluster seed node: akka://credtrack-ai@127.0.0.1:2551
- Split-brain resolver: SplitBrainResolverProvider (downing minority side)

- AnalyticsWorkerActor entity type key: "analytics-worker" sharded by userId

11. Course Topic Coverage

The following table maps each CSYE 7374 course topic to its concrete implementation in the CredTrack AI agent:

Course Topic	Implementation in CredTrack
Understanding Large Language Models	Llama 3.1 via Ollama — prompt engineering for structured JSON output, confidence scoring, JSON regex extraction
Working with Text Data	Jsoup HTML stripping, whitespace normalisation, 4,000-char truncation, URL extraction before stripping
Introduction to Actor Model with Akka	18 typed actors; sealed trait message protocols; no shared mutable state; all I/O via pipeToSelf
Fault Handling with Akka	restartWithBackoff on PersistentStatementLedgerActor (2s→60s) and TransactionRouterActor (1s→30s); nested strategies on UtilityBillRouterActor; historyId suppression on LLM failure
Akka Clusters	Cluster-mode deployment; seed node 2551; SplitBrainResolverProvider for partition handling; ClusterSharding on AnalyticsWorkerActor by userId; horizontally scalable by adding seed-node addresses
Spring AI	ChatClient.Builder injection; Ollama provider; format=json; provider-swappable in one config line
Integrating LLM and Akka Actor Model	pipeToSelf pattern used in 13+ actor methods; Java 21 virtual threads for LLM I/O; LLM results routed back as typed messages
Streams and Persistence	EventSourcedBehavior (PersistentStatementLedgerActor); StatementClosedEvent journal; full state replay on crash

12. Above and Beyond

12.1 Dual-Pipeline Architecture

The system runs two completely independent coordinator pipelines in the same actor system — one for credit card statements and one for utility bills — each with their own init/incremental lifecycle, supervision trees, and LLM extraction prompts. Both pipelines share the same virtual thread executor and Spring AI ChatClient, but their actor state is fully isolated.

12.2 Confidence-Gated Extraction

The LLM is asked to self-report its confidence (0–1) on every extraction call. Statements and utility bills below 0.7 confidence are silently discarded. Transaction alerts use a stricter 0.90 threshold. This prevents low-quality extractions from polluting the database, which is especially important for transaction alerts where short email snippets can be ambiguous.

12.3 Java 21 Virtual Threads for LLM Concurrency

All blocking I/O runs on a Java 21 virtual thread executor (`Executors.newVirtualThreadPerTaskExecutor()`). Virtual threads are extremely lightweight — the JVM can sustain millions concurrently. This means the system can have hundreds of in-flight LLM calls simultaneously (one per email across all users) without any thread pool sizing, without blocking OS threads, and without back-pressure configuration. It is a fundamentally different scaling model from a traditional thread pool executor.

12.4 Zero Data-Loss Retry Architecture

The combination of historyId suppression on LLM failure + PostgreSQL unique constraint on `gmail_message_id` + event-sourced ledger creates a three-layer guarantee: any email that fails to process will be retried; any statement that is successfully written will not be duplicated on retry; and any state built in the ledger actor will survive a crash. The system tolerates LLM outages, network failures, and process crashes without any manual intervention or data repair.

13. Conclusion

The CredTrack AI Agent demonstrates that Akka Typed, Spring AI, and a locally-hosted LLM can be composed into a production-grade intelligent pipeline that is resilient, concurrent, and maintainable. The key design decisions — `pipeToSelf` for non-blocking LLM calls, historyId suppression for zero data-loss retry, confidence thresholds for extraction quality, and cluster sharding for per-user isolation — each address a real failure mode that would occur in a deployed system.

The actor model proved to be the right abstraction for this problem. Each stage of the pipeline (fetch, extract, validate, write) maps naturally to one actor type, the supervision hierarchy handles failures declaratively without any try-catch scattered through business logic, and the typed message protocols made the system easy to reason about and extend. Adding a new bank or a new email format requires only a new search query string and a new regex pattern in `PaymentExtractorActor` — no changes to the coordinator or the LLM prompt.

Future enhancements include: migrating the Akka journal from in-memory to Cassandra for durable event sourcing across restarts, adding a RAG-based conversational interface (as originally proposed) using a vector database to retrieve relevant transactions as context, and scaling the cluster to multiple nodes for true distributed deployment — the sharding and supervision infrastructure is already in place and requires only seed-node configuration to activate.